

This assignment may take more time than previous ones. Start early.

The descriptions may feel abstract at first. Be sure to read the provided starter code (including the test code) to better understand the system. The goal of this assignment is to get you fluent in using classes in C++. The core topics here are polymorphism, interfaces, and inheritance.

Survivor Camp Simulator

We'll be making an apocalypse survivor game simulator.

Survivor Camp

The SurvivorCamp class manages each round in the simulation. It has a function SimulateCamp() that returns the number of turns before the simulation ends. Each turn, it should:

1. Tell the held Survivor* to take their turn, if alive.
2. Check if the current Mutant is dead -- remove it from the vector if so, or otherwise allow that Mutant to take its turn.
3. Increment the number of turns by one.

You can add a Mutant to the camp's encounter list by calling the AddMutant(ECMutant* mutant) function. This adds the mutant to the back of the vector. Mutants take actions in vector order (i.e., the mutant at index 0 acts first).

Survivors

A Survivor will perform one attack every turn. The exact behavior depends on how the specific subclass implements the TakeTurn function.

- a. The key interface function of Survivor is TakeTurn. This function takes a mutant object and will attack it by calling that mutant's ReceiveAttack function. Each subclass may include additional unique logic.
- b. A concrete subclass of Survivor, Scavenger, is provided. This survivor simply attacks the first mutant in the vector each turn.

Brawler

The Brawler is a subclass of Survivor. It acts like a Scavenger, but every time it kills a mutant, it gains +2 damage for future attacks.

Acrobat

The Acrobat is another subclass of Survivor. It acts like a Scavenger, but every four turns, it regains the ability to dodge the next incoming attack. The Acrobat starts with the dodge ability available on turn 0 and again on turn 4, 8, etc.

Important nuance: if the Acrobat eliminates mutants fast enough to avoid incoming attacks, it retains its dodge ability until needed (it doesn't waste dodges). However, even after long periods without being attacked, it will still regain dodges every four turns.

Medic

The Medic is a subclass of the Survivor. Like the Scavenger, it attacks the first mutant every turn but is equipped with a medical ability: every three turns, the Medic heals itself, regaining a portion of its lost health and recovering from any status afflictions (like poison). Like the Acrobat, the Medic starts by healing itself then heals itself again on turn 3, 6, etc.

Note: to make things simple, a medic can heal a static 5 health points, regardless of its initial health amount. Yes, that means its health can grow greater than the input health. Medic is chill like that.

Also note: if medic is poisoned and can heal itself, it heals itself *before* taking poison damage, thus removing the effect *and* gaining 5 health.

Mutants

A Mutant is a class hierarchy of hostile creatures the Survivors must defeat! Mutants also act every turn, attacking or performing special behaviors.

Zombie

The Zombie is a basic mutant. It attacks every turn and is implemented for you.

Replicator

The Replicator is a subclass of Mutant. In addition to attacking every turn, it creates an exact copy of itself every other turn. The copy inherits the state from the turn it was duplicated (e.g., current health). Replicators created by other replicators can also replicate themselves.

Replicator clones are added to the end of the remainingMutants vector in the SurvivorCamp, even if the Replicator is part of a pack.

Hint: The Replicator will need a pointer to its SurvivorCamp to add new mutants to the encounter list.

Spitter

The Spitter is a subclass of Mutant that fights in a more indirect way.

- Every other turn (starting on turn 0), the Spitter will apply an acid effect to the Survivor, dealing damage-over-time for two turns.
 - At the start of the next turn for the Survivor, before they attack, they will take 2 damage. Then, at the start of their next turn, they will take 2 more damage. After the 2nd time taking damage, they will be healed. This results in 4 damage total before healing.
- On alternating turns, it will perform a regular attack like a Zombie.

Important: Status effects like the acid/poison do not stack. For example, if multiple Spitters (or a Spitter inside a MutantPack) apply the acid effect in the same round, the Survivor will only suffer from a single instance of the poison damage, not multiple overlapping stacks. Being poisoned again while already poisoned does not extend the duration nor the intensity of the poison's effects.

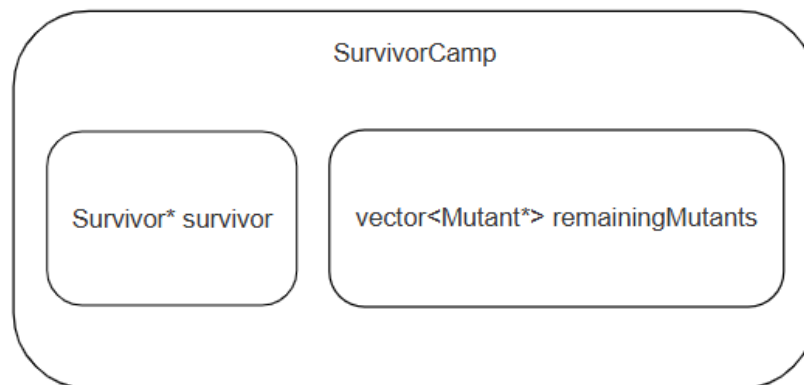
Mutant Pack

The MutantPack is a subclass of Mutant. It represents a group of mutants fighting as a unit.

- When the MutantPack attacks, every mutant inside will attack the Survivor.
- When the MutantPack is attacked, only the first mutant inside takes damage.
- The MutantPack is only defeated when all contained mutants are dead.

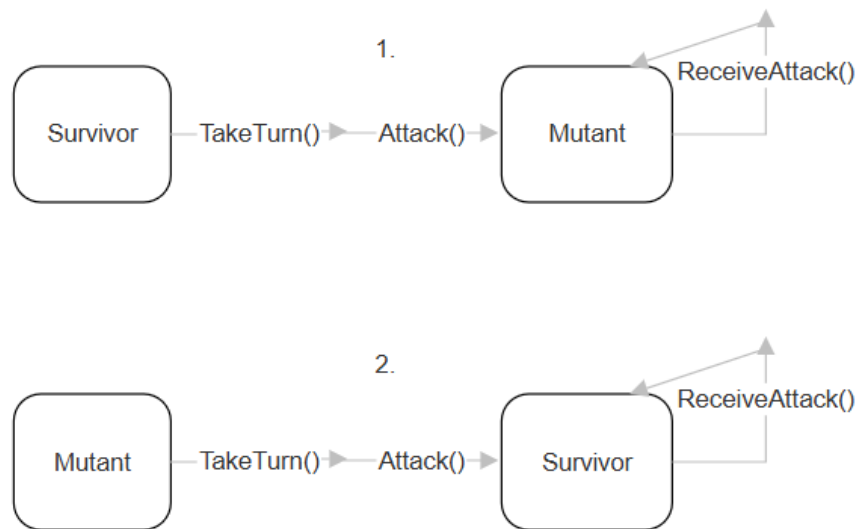
Note: When the Brawler kills an individual mutant inside a MutantPack, it does not gain bonus damage. The bonus is only gained when the entire pack is defeated.

Also: If a member of a pack kills the survivor (survivor health ≤ 0) but there are still mutants in the pack that have not attacked yet this turn, they will **still** attack the survivor. Survivor's health will continue to decrease into the negatives. We don't check if we should end a simulation until the full turn is over.



Internal structure of SurvivorCamp

SimulateTurn()



General flow of one call to SimulateTurn()

Conclusion

This system might seem large at first glance. Tests have been provided to help you get started and to familiarize you with how the Gradescope tests will function.

Approach this assignment incrementally and ensure one part works before moving to the next. As the course progresses into broader topics such as Software Development and Object-Oriented Design, you will work on larger codebases. This assignment is designed to help you build that skill.

Start on this assignment early and be sure to post any questions on Piazza, if you get stuck, please reach out to brendan.barnett@uconn.edu or allen.tran@uconn.edu.